

SK네트웍스 Family AI 과정 21기

모델링 및 평가 LLM 활용 소프트웨어

산출물 단계	모델링 및 평가
평가 산출물	LLM 활용 소프트웨어
제출 일자	2026.03.19
깃허브 경로	https://github.com/skn21-final-1team
작성 팀원	우재현

목차

1. 시스템 개요
2. 시스템 구성 요소
3. 코드 구조
4. 주요 코드
5. 프롬프트 최적화
6. 보안사항 고려
7. 테스트 시나리오
8. 평가

1. 시스템 개요

본 소프트웨어는 웹 문서 수집 → 정제 → 임베딩 → 벡터 저장 → RAG 기반 답변 생성까지의 전체 파이프라인을 제공하는 LLM 활용 노트북 시스템입니다.

LangGraph 기반 RAG 에이전트가 의도 분류, 벡터 검색, Cross-Encoder 리랭킹, LLM 답변 생성을 오케스트레이션하며, 프롬프트 최적화와 deepeval 평가 체계를 통해 답변 품질을 관리합니다.

2. 시스템 구성 요소

데이터 수집: Frontend(Next.js) / Extension(React+CRXJS)

→ Backend(FastAPI) → Data Service(FastAPI)

→ RunPod(vLLM 정제/요약, bge-m3 임베딩) → PostgreSQL+pgvector

RAG 답변: Frontend(Next.js) → Backend(FastAPI+LangGraph)

1) 의도 분류(LLM: simple/complex/casual 3-way 분기)

2) 복합 질문 분해(LLM, complex만)

3) PostgreSQL+pgvector 벡터검색

4) RunPod(bge-reranker-v2-m3) 리랭킹

5) 소스 자료 + 대화 이력 기반 프롬프트 구성

6) OpenAI API(GPT-4o-mini) / RunPod(EXAONE) 답변 생성

7) Frontend(SSE 스트리밍)

사용 모델

구성 요소/구분	모델	위치/경로
본문정제 vLLM	meta-llama/Llama-3.1-8B-Instruc	post-processing
임베딩	BAAI/bge-m3	embedding
리랭커	BAAI/bge-reranker-v2-m3	rerank
답변 생성1	GPT-4o-mini, GPT-4o, GPT-5-mini	-
답변 생성2	LGAI-EXAONE/EXAONE-4.0-32B-FP8	exaone
평가 Judge	GPT-4o-mini (deepeval)	-

3. 코드 구조

Data Service (데이터 파이프라인)

```
data/
├── crawl/                # 웹 크롤링 (Trafilatura + Playwright)
│   ├── client.py        # 하이브리드 스크래핑 클라이언트
│   └── preprocess.py    # MarkdownPreprocessor
├── llm/                 # vLLM 연동 (정제/요약)
│   ├── client.py        # refine(), summarize() 호출
│   └── prompts.py       # 정제/요약 프롬프트 템플릿
├── chunk/               # 청킹 전략
│   └── strategies.py     # HierarchicalPrependChunker
├── embed/               # 임베딩 생성 (bge-m3)
│   └── service.py        # embed_texts()
├── services/
│   └── crawl_pipeline.py # 메인 파이프라인 (크롤→정제→청킹→임베딩)
├── crud/ · models/ · schemas/ # DB 레이어
└── api/                 # FastAPI 엔드포인트 (/crawl)
```

Backend (RAG 오케스트레이션)

backend/

- | — agent/ # RAG 에이전트 핵심 모듈
 - | | — edges/ # 조건부 라우팅 로직
 - | | | — route_by_intent.py # 의도 기반 3-way 라우팅
 - | | | — route_by_sources.py # 검색 결과 기반 재시도 라우팅
 - | | — model/ # AI 모델 레이어
 - | | | — llm_factory.py # LLM 팩토리 (GPT-4o-mini, EXAONE)
 - | | | — embedding.py # bge-m3 임베딩 인스턴스
 - | | | — reranker.py # bge-reranker-v2-m3 리랭커
 - | | — nodes/ # 그래프 처리 노드
 - | | | — classify_intent.py # 3분류 의도 분류기
 - | | | — decompose_query.py # 복합 질문 분해
 - | | | — rewrite_query.py # 쿼리 재작성
 - | | | — retrieve_sources.py # 벡터 검색 + 리랭킹
 - | | | — generate_answer.py # 답변 생성 (QA + Casual)
 - | | | — save_chat.py # 대화 이력 저장
 - | | — prompts/ # 프롬프트 템플릿
 - | | | — classify.py # 의도 분류 프롬프트
 - | | | — rewrite.py # 쿼리 재작성/분해 프롬프트
 - | | | — chat.py # QA/Casual 답변 프롬프트
 - | | — graph.py # LangGraph 워크플로우 정의
 - | | — state.py # QASState 타입 정의
- | — core/ # 설정 및 공통 모듈
 - | | — config.py # 환경변수 설정 (pydantic-settings)
 - | | — exceptions/ # 커스텀 예외 처리
- | — eval/ # 평가 프레임워크
 - | | — evaluate.py # deepeval A/B 평가 스크립트
 - | | — baseline_graph.py # 베이스라인 그래프 (비교용)
 - | | — results/ # 평가 결과 JSON 파일
- | — services/ # 비즈니스 로직
 - | | — agent.py # SSE 스트리밍 서비스
- | — api/ # FastAPI 엔드포인트
- | — crud/ # DB CRUD 연산
- | — db/ # 데이터베이스 설정
- | — models/ # SQLAlchemy 모델
- | — schemas/ # Pydantic 스키마
- | — alembic/ # DB 마이그레이션
- | — main.py # FastAPI 앱 엔트리포인트

모듈화 원칙

- 단일 책임: 각 노드 파일은 하나의 처리 단계만 담당
- 팩토리 패턴: `llm\_factory`로 모델 인스턴스 생성을 중앙화
- 프롬프트 분리: 프롬프트 텍스트를 `prompts/` 디렉토리에 독립 관리
- 상태 기반: `QASState` TypedDict로 노드 간 데이터 흐름 타입 안전성 보장
- 환경변수 분리: `core/config.py`에서 `.env` 기반 설정 관리 (`@lru\_cache`)

4. 주요 코드

4.1 예외 처리

classify_intent - 분류 실패 시 안전한 fallback

```
1 # agent/nodes/classify_intent.py
2 def classify_intent(state: QASState, config: RunnableConfig) -> dict[str, str]:
3     try:
4         llm = llm_factory.get_llm(config)
5         if llm_factory.supports_structured_output(config):
6             response = llm.with_structured_output(RouteByIntent).invoke(messages)
7             return {"intent": response.intent}
8         else:
9             # structured output 미지원 모델: 텍스트에서 파싱
10            response = llm.invoke(messages)
11            return {"intent": _parse_intent_from_text(response.content)}
12    except Exception as e:
13        print("classify_intent error", e)
14    return {"intent": "simple"} # fallback: simple (검색 후 안내 가능)
```

- structured output 미지원 모델(EXAONE 등)은 텍스트 파싱으로 대체
- 에러 시 `"simple"` 반환 (casual보다 안전 - 검색 후 없으면 안내 가능)

decompose_query - 파싱 실패 시 원본 질문 사용

```
1 # agent/nodes/decompose_query.py
2 async def decompose_query(state, config):
3     try:
4         if llm_factory.supports_structured_output(config):
5             response = await llm.with_structured_output(DecomposedQueries).ainvoke(messages)
6             return {"search_queries": response.queries}
7         else:
8             response = await llm.ainvoke(messages)
9             return {"search_queries": _parse_queries_from_text(response.content, state["question"])}
10    except Exception:
11        return {"search_queries": [state["question"]]} # fallback: 원본 질문
```

- JSON, 번호 목록, 줄바꿈 등 다양한 형식 파싱 시도
- 최종 실패 시 원본 질문으로 검색 (기능 중단 방지)

reranker - API 에러/타임아웃 처리

```
1 # agent/model/reranker.py
2 async def rerank(self, query, documents, top_k=None):
3     try:
4         async with httpx.AsyncClient(timeout=60) as client:
5             response = await client.post(self.CONFIG["url"], ...)
6             response.raise_for_status()
7             # min_score 미만 필터링
8             filtered = [v for v in output if v.get("score", 0) >= self.CONFIG["min_score"]]
9             return [v.get("document", "") for v in filtered]
10    except httpx.HTTPStatusError as e:
11        print(f"Reranker API error: {e.response.status_code} - {e.response.text}")
12        return []
13    except httpx.ReadTimeout:
14        print("Reranker timeout: 응답 대기 시간 초과")
15        return []
```

- HTTP 4xx/5xx 에러, ReadTimeout 각각 분리 처리
- 실패 시 빈 리스트 반환 → `route\_by\_sources`에서 재시도 또는 graceful degradation

decompose_query - 파싱 실패 시 원본 질문 사용

```
1 # agent/nodes/decompose_query.py
2 async def decompose_query(state, config):
3     try:
4         if llm_factory.supports_structured_output(config):
5             response = await llm.with_structured_output(DecomposedQueries).ainvoke(messages)
6             return {"search_queries": response.queries}
7         else:
8             response = await llm.ainvoke(messages)
9             return {"search_queries": _parse_queries_from_text(response.content, state["question"])}
10    except Exception:
11        return {"search_queries": [state["question"]]} # fallback: 원본 질문
```

- JSON, 번호 목록, 줄바꿈 등 다양한 형식 파싱 시도
- 최종 실패 시 원본 질문으로 검색 (기능 중단 방지)

평가 스크립트 - 배치 실패 시 **resume** 지원

```
1 # eval/evaluate.py
2 def _run_batch_with_retry(batch, metrics, batch_num, total_batches, batch_delay):
3     max_retries = 1
4     for attempt in range(1, max_retries + 1):
5         try:
6             return evaluate(test_cases=batch, metrics=metrics)
7         except Exception as e:
8             if attempt < max_retries:
9                 time.sleep(batch_delay * 2) # 재시도 전 대기
10            else:
11                print(f" 배치 {batch_num} 건너뛴 (재시도 소진)")
12    return None
```

- 배치 평가 실패 시 1회 재시도
- 중간 결과를 `\_partial\_\*.json`에 저장하여 크래시 복구 가능
- `--resume` 옵션으로 이전 결과 이어서 평가

Data 파이프라인 - 2-Phase 처리 + 체크포인트 재시도

```

1  # data/services/crawl_pipeline.py
2  async def process_pipeline(source_map: dict[str, int]) -> None:
3      """Phase 1: 크롤링(순차) → Phase 2: LLM+임베딩(asyncio 병렬)
4      # Phase 1 — 외부 사이트 부하 방지를 위해 순차 크롤링
5      for url, source_id in source_map.items():
6          result = await _crawl_single(url, source_id)
7
8      # Phase 2 — asyncio.gather + Semaphore로 병렬 처리
9      success = await _run_parallel_processing(crawled)

```

```

1  # data/services/crawl_pipeline.py
2  async def _process_content(source_id: int, raw_content: str) -> int:
3      """전처리 → vLLM 정제 → vLLM 요약 → 청킹 → 임베딩 → PGVector 적재."""
4      # 재시도 시 이미 완료된 단계 건너뛰기 (DB 상태 확인)
5      with get_db_context() as db:
6          source = get_source_by_id(db, source_id)
7          has_refined = source and source.refined
8          has_summary = has_refined and source.summary
9
10         if has_summary:           # embed부터 실행
11             ...
12         elif has_refined:         # summarize부터 실행
13             summary_text = await summarize(refined_text)
14         else:                     # 전체 파이프라인 실행
15             preprocessed = MarkdownPreprocessor.run(raw_content)
16             refined_text = await refine(preprocessed)
17             summary_text = await summarize(refined_text)
18
19         chunks = [c.content for c in chunk_text_only(refined_text)]
20         embeddings = await embed_texts(chunks)
21         bulk_create_page_data(source_id=source_id, chunks=chunks, embeddings=embeddings)

```

- Phase 1(크롤링)은 외부 사이트 부하 방지를 위해 순차 실행
- Phase 2(LLM +임베딩)는 `asyncio.Semaphore`로 병렬 처리 (MAX_WORKERS 제한)
- 재시도 시 DB 상태 확인으로 완료된 단계 건너뛰기 (체크포인트 최적화)
- 각 단계별 커스텀 예외 (`RefineError`, `SummarizeError`, `EmbedError` 등) 분리

5. 프롬프트 최적화

5.1 데이터 전처리 프롬프트

크롤링된 웹 문서를 벡터 DB에 저장하기 전, vLLM을 통해 정제와 요약을 수행합니다.

정제(Refine) 프롬프트

[정제 규칙 — 최우선: 축약 금지]

- 원본 본문의 모든 문장, 문단, 데이터를 그대로 유지
- 정제 후 글자 수는 반드시 원본의 **90% 이상**
- 제거 대상: 스크립트 잔여물, UI/네비게이션 텍스트, 깨진 문자, 중복 패턴, 푸터/저작권
- 위 항목 외에는 단어 하나도 삭제하거나 바꾸지 않음

요약(Summarize) 프롬프트

[요약 규칙]

- 페이지의 주제와 성격을 **4~5문장**으로 소개 (본문 자체를 요약하지 않음)
- 본문 언어에 맞춰 작성 (영어→영어, 한국어→한국어, 혼합→한국어)

최적화 포인트

- 축약 금지 원칙: "**90% 이상 유지**" 규칙으로 청크에 충분한 맥락 보존 → 검색 품질 향상
- 노이즈만 제거: UI 텍스트, 스크립트 잔여물 등 검색에 방해되는 요소만 선별 제거
- 페이지 소개 방식: 본문 요약이 아닌 주제/성격 소개로, 소스 미리보기 용도에 적합

5.2 의도 분류 프롬프트

3단계 분류 기준을 명시적 예시와 함께 제공

"simple" - 단일 주제 직접 질문/애매한 질문

예: "머신러닝이 뭐야?", "그거 장점이 뭐야?"

"complex" - 비교/분석/다단계 추론

예: "A와 B의 차이점은?", "장단점을 정리해줘"

"casual" - 소스 참조 불필요한 대화

예: "안녕", "너 이름이 뭐야?"

최적화 포인트

- fallback을 `simple`로 설정하여 검색 후 안내 가능하게 함 (casual로 빠지면 소스 검색 불가)

5.3 쿼리 재작성 프롬프트

1. 대명사/생략된 주어를 대화 이력에서 복원
2. 핵심 키워드 추출, 명사 중심 구문 변환
3. 질문 의미 변경 금지
4. 재시도 시: 이전 쿼리와 다른 관점/동의어 사용

최적화 포인트

- 재시도(`is\_retry=True`) 시 이전 검색 쿼리를 참조하여 다른 관점의 쿼리 생성

5.4 QA 답변 프롬프트

핵심 원칙

- 근거 기반 답변: 소스 자료에 근거
- 소스에 없는 내용 추측/지어내기 금지
- [소스 N] 형태로 인용 표기

소스 부족 시 대응

- 전혀 없음: "해당 내용을 소스에서 찾을 수 없습니다"
- 부분적: 관련 부분만 답변 + 나머지 안내
- 무관: "관련 내용을 찾지 못했습니다"

최적화 포인트

- Structured Context Injection: 소스에 `[소스 1]`, `[소스 2]` 번호 부여
- Strict Grounding: 소스 외 내용 추측 금지로 Faithfulness 향상
- Graceful Degradation: 소스 부족 시 3단계 대응 전략

5.5 프롬프트 실험 이력

실험	변경 내용	Faithfulness	Answer Relevancy	결과
baseline	기본 프롬프트	0.916	0.773	기준선
실험1	"배경지식, 일반상식, 추론 포함 금지" 강화	0.889	0.788	답변 거부 발생 → 롤백
실험2(최종)	소스 인용 + graceful degradation	0.919	0.856	채택

실험 요약

baseline 프롬프트에 소스 인용, "배경지식, 일반상식, 추론을 포함하지 마세요" 조건과 소스인용, graceful degradation을 추가.

소스 외 정보 사용을 완전히 차단하려는 의도였으나, LLM이 소스가 있음에도 답변을 거부하는 사례가 3건 발생했으며, 소스 내용을 재구성하는 과정에서도 "추론"으로 판단하여 답변을 회피한 것으로, 과도한 제약이 오히려 Faithfulness를 하락(0.916→0.889)시킴.

실험2에서는 배경지식, 일반상식, 추론 포함금지 문구를 삭제하되, 소스인용과 graceful degradation을 유지함으로 3단계 대응을 적용하였다.

baseline → 최종: 주요 변경점

항목	baseline	최종
소스 포맷	`---` 구분자로 소스 나열	`[소스 1]`, `[소스 2]` 번호 부여 (Structured Context Injection)
소스 부족 대응	단일 안내 메시지	3단계 대응 (전혀 없음 / 부분적 / 무관)
인용 표기	없음	`[소스 N]` 형태로 답변 내 근거 표기 의무화
답변 구조화	기본 마크다운 안내	굵게/목록/코드 블록 등 구체적 포맷 가이드

6. 보안 고려 사항

6.1 API Key 및 환경변수 관리

- API Key는 코드에 하드코딩하지 않고, `.env` + `pydantic-settings`로 로드 (`core/config.py`)
- `.env`는 `.gitignore`에 등록되어 원격 저장소에 업로드되지 않음

환경변수	용도
OPENAI_API_KEY	GPT-4o-mini API 호출
RUNPOD_API_KEY	임베딩/리랭커/EXAONE 서빙 인증
EMBEDDING_MODEL_URL	bge-m3 임베딩 서비스 엔드포인트
RERANKER_MODEL_URL	bge-reranker-v2-m3 서비스 엔드포인트
CUSTOM_LLM_MODEL_URL	EXAONE LLM 서비스 엔드포인트
DATABASE_URL	PostgreSQL 동기 연결
ASYNC_DATABASE_URL	PostgreSQL 비동기 연결 (벡터 검색)

6.2 데이터 격리

- 벡터 검색 시 `notebook_id` 기반 소스 필터링으로 사용자 간 데이터 격리
- `get_active_source_ids()`로 비활성화된 소스 자동 제외

7. 테스트 시나리오

시나리오 1: 단순 질문 (simple)

질문: "Notion AI가 제공하는 핵심 기능들은 무엇인가요?"

의도 분류: simple

경로: `classify_intent` → `retrieve_sources` → `generate_answer` → `save_chat`

응답 예시:

"Notion AI가 제공하는 핵심 기능은 다음과 같습니다.

1. **Notion 에이전트**: 페이지, 문서, 작업에서 직접 작업 [소스 1]
2. **AI 노트**: 회의 기록 요약 및 실무 중심 정리 [소스 1]
3. **기업 통합 검색**: 여러 앱에서 정보 검색 [소스 2]

..."

→ 벡터 검색 + 리랭킹 + 소스 인용 답변 확인

시나리오 2: 복합 질문 (complex)

질문: "LightRAG가 기존 RAG의 어떤 한계를 해결하고, 핵심 기술의 특징은?"

의도 분류: complex

경로: `classify_intent` → `decompose_query` → `retrieve_sources` → `generate_answer`

분해된 쿼리:

- "LightRAG가 해결하는 기존 RAG 한계점"
- "LightRAG 핵심 기술 특징"

→ 질문 분해 + sub-query별 검색/리랭킹 + 통합 답변 확인

시나리오 3: 일상 대화 (casual)

질문: "안녕하세요!"

의도 분류: casual

경로: `classify_intent` → `casual_answer` → `save_chat`

응답 예시:

"안녕하세요! 반갑습니다. 노트북에 저장된 소스 자료에 대해 궁금한 점이 있다면
편하게 질문해 주세요."

→ 소스 검색 없이 자연스러운 대화 응답 확인

시나리오 4: 검색 결과 0건 재시도

질문: "PCCP 자격증의 난이도별 특징은?"

의도 분류: simple

1차 경로: `retrieve_sources` → 0건 → `rewrite_query` (동의어/다른 관점)

2차 경로: `retrieve_sources` → N건 → `generate_answer`

→ 재시도 메커니즘으로 검색 실패 복구 확인

시나리오 5: 소스 부족 시 **Graceful Degradation**

질문: "양자컴퓨팅의 원리를 설명해줘" (소스에 없는 주제)

경로: retrieve_sources → 0건 → rewrite_query → 0건 → generate_answer

응답 예시:

"해당 내용을 소스에서 찾을 수 없습니다.

관련 자료를 노트북에 추가해 보세요."

→ 소스 없는 질문에 대한 안전한 답변 확인 (hallucination 방지)

8. 평가

8.1 평가 프레임워크

- 도구: deepeval (LLM-as-Judge)
- 평가 모델: GPT-4o-mini
- 테스트셋: 50건 (단일 소스 복합 질문 20건, 복합-비관련 15건, 복합-관련 15건)
- 배치: 5건/배치, 60초 간격, 1회 재시도, resume 지원

8.2 평가 메트릭

메트릭	설명	측정 대상
Contextual Precision	검색된 소스 중 관련 소스 비율	검색 정밀도
Contextual Recall	필요한 소스를 얼마나 찾았는가	검색 재현율
Contextual Relevancy	검색된 소스가 질문에 얼마나 관련 있는가	검색 관련성
Faithfulness	답변이 소스에 충실한가 (hallucination 없는가)	답변 신뢰성
Answer Relevancy	답변이 질문에 적절히 응답하는가	답변 품질

Contextual Precision, Contextual Recall, Contextual Relevancy는 ground_truth 작성 내용에 따라 성능이 바뀌기 때문에, 기준치를 충족 여부로 판단한다. 각각(0.8, 0.8, 0.3)

Faithfulness와 Answer Relevancy의 값을 가장 중요한 평가 요소로 정하여 평가를 진행한다.

8.3 그래프 실험 결과 (gpt-4o-mini)

메트릭	BASELINE	requery_prompt	strict_grounding	rewrite skip	sub-query rerank	sub-query +5제한
C.Precision	0.954	0.831	0.809	0.857	0.882	0.898
C.Recall	0.919	0.802	0.810	0.826	0.950	0.956
C.Relevancy	0.462	0.360	0.398	0.392	0.370	0.393
Faithfulness	0.916	0.867	0.889	0.919	0.881	0.877
Answer Relevancy	0.773	0.811	0.788	0.856	0.933	0.923

Sub-query rerank가 Answer Relevancy에서 가장 높은 점수를 보였으나 faithfulness에서 baseline보다 낮은 점수를 보였다.

rewrite skip은 baseline대비 Faithfulness와 Answer Relevancy 모두 더 나은 점수를 보였으며, faithfulness 에서 가장 높은 수치를 보였다.

실험별 변경 사항

실험변경 내용	설명
BASELINE	2분류(question/casual), 직접 검색 기본 프롬프트
requery_prompt	3분류, decompose_query, rewrite_query, reranker 항상 실행, 소스 인용 프롬프트
strict_grounding	requery_prompt + 프롬프트 강화 ("배경지식/추론 금지") → 답변 거부 발생으로 롤백
rewrite skip	requery_prompt + simple→retrieve_sources 직행 (rewrite 스킵)
sub-query rerank	rewrite skip + 복합 질문 sub-query별 개별 리랭킹, min_results=2 보장

sub-query+cap5	sub-query rerank + 최종 소스 5건 제한
----------------	--------------------------------

8.4 모델 비교 결과(requery_prompt 기준)

메트릭	EXAONE	gpt-4o-mini	gpt-4o	gpt-5-mini
C.Precision	0.709	0.843	0.882	0.855
C.Recall	0.636	0.834	0.886	0.848
C.Relevancy	0.334	0.395	0.380	0.392
Faithfulness	0.891	0.898	0.916	0.903
Answer relevancy	0.655	0.804	0.935	0.861

모델별 분석

- gpt-4o: 전 메트릭 1위. Faithfulness 0.916, AR 0.935로 최고 품질. 비용 높음
- gpt-5-mini: gpt-4o-mini 대비 소폭 개선 (AR +0.057). 비용 대비 판단 필요
- gpt-4o-mini: 안정적 가성비 모델. 프로덕션 기본값으로 채택
- EXAONE: AR 0.655로 답변 품질 부족. Contextual 지표도 전반적으로 낮음

최종 선택: rewrite skip & GPT-4o-mini

rewrite skip은 Faithfulness가 가장 높은 점수를 보임으로 소스 충실성 유지에 큰 점수를 부여했으며, 동시에 answer relevancy(답변품질) 또한 baseline 대비 약 10.7% 향상 했기에 채택.

Gpt-4o와 Gpt-5-mini는 Faithfulness와 Answer Relevancy 모두 gpt-4o-mini보다 높은 점수를 보였으나, 비용적인 측면에서 제외되었다.

Gpt-4o-mini를 기본 모델로 사용하는 대신, 모델 선택 기능을 작성하여 exaone과 gpt-5.4-mini 등 다양한 모델을 적용할 수 있게 서비스를 개선하는 방향을 진행하였다.

8.5 최종 파이프라인 반영 내용

구성요소	역할
3-way 의도 분류	질문을 simple/complex/casual로 분류하여 불필요한 처리 경로 제거 (casual은 검색 스킵)
decompose_query	complex 질문을 sub-query로 분해하여 다중 소스 검색 정확도 향상
rewrite_query	대명사 복원·키워드 추출로 검색 쿼리 최적화 (complex만 실행, simple은 스킵)
reranker 항상 실행	벡터 검색 결과를 Cross-Encoder로 재정렬, min_score 기반 저품질 소스 필터링
소스 인용 프롬프트	[소스 N] 형태로 답변 내 근거 표기 의무화, 3단계 소스 부족 대응
simple→retrieve 직행	simple 질문은 rewrite_query를 건너뛰고 바로 벡터 검색 실행 (불필요한 LLM 호출 제거)